Visualización de Series de Tiempo **(NGSI-LD)**

Gestión de Datos en Entornos Inteligentes (XDEI)

NGSI-LD
Time Series Data

Visualización de Series de Tiempo (NGSI-LD)

Time Series Data (NGSI-LD)

- Tutorial

```
git clone https://github.com/FIWARE/tutorials.Time-Series-Data.git
cd tutorials.Time-Series-Data
git checkout NGSI-LD
```

- En *docker-compose/common.yml:*
  - Modificamos el apartado del *iot-agent:*

```
deploy:
    resources:
      limits:
        memory: 1024M
        cpus: '2.0'
      reservations:
        memory: 512M
        cpus: '1.0'
```

  - Modificamos el apartado *grafana:*

```
image: grafana/grafana:11.0.4
```

  - A continuación:

```
./services start
```

Time Series Data (NGSI-LD)
Suscripciones de QL al CB

- **QL se suscribe al CB para recibir las actualizaciones** de los datos de contexto y almacenarlas como histórico
- Suscripción a los eventos del **sensor de llenado**

```
curl -L -X POST 'http://localhost:1026/ngsi-ld/v1/subscriptions/' \
    -H 'Content-Type: application/ld+json' \ -H 'NGSILD-Tenant: openiot’ \
    --data-raw '{
        "description": "Notify me of all feedstock changes",
        "type": "Subscription",
        "entities": [{"type": "FillingLevelSensor"}],
        "watchedAttributes": ["filling"],
        "notification": {
            "attributes": ["filling", "location"], "format": "normalized",
            "endpoint": {
                "uri": "http://quantumleap:8668/v2/notify",
                "accept": "application/json",
                "receiverInfo": [ {
                     "key": "fiware-service", "value": "openiot" } ] } },
        "@context": "http://context/ngsi-context.jsonld" }'
```

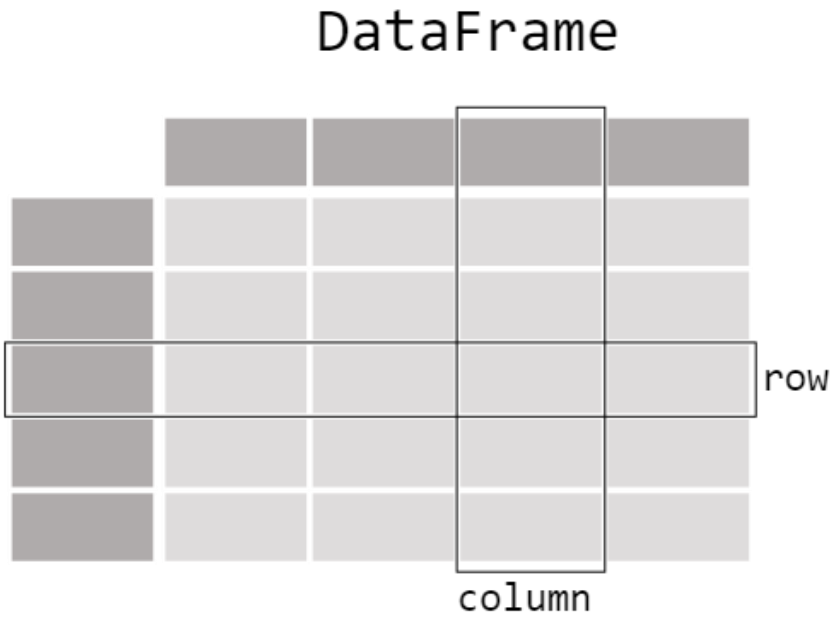
Time Series Data
Suscripciones de QL al CB (II)

- Suscripción a los eventos de **lecturas GPS**

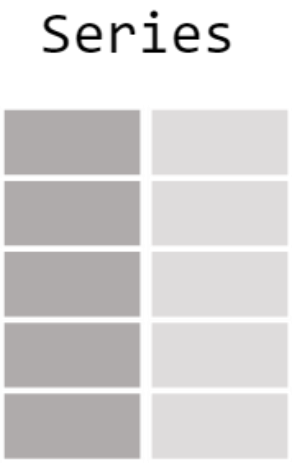
```
curl -L -X POST 'http://localhost:1026/ngsi-ld/v1/subscriptions/' \
-H 'Content-Type: application/ld+json' -H 'NGSILD-Tenant: openiot' \
--data-raw '{
  "description": "Notify me of animal locations",
  "type": "Subscription",
  "entities": [{"type": "Device"}],
  "watchedAttributes": ["location", "status", "heartRate"],
  "notification": {
    "attributes": ["location", "status", "heartRate"],
    "format": "normalized",
    "endpoint": {
      "uri": "http://quantumleap:8668/v2/notify",
      "accept": "application/json",
      "receiverInfo": [ {
          "key": "fiware-service",
          "value": "openiot" } ]
    }
  },
   "throttling": 10,
   "@context": "http://context/ngsi-context.jsonld" }'
```

DataFrame

- **Tabla en memoria**
- **Multitud de operaciones**
- Operaciones **tipo SQL**



- Cada columna es una Series



DataFrame (II)

- Los datos en un DataFrame están organizados igual que en un diccionario. De hecho:

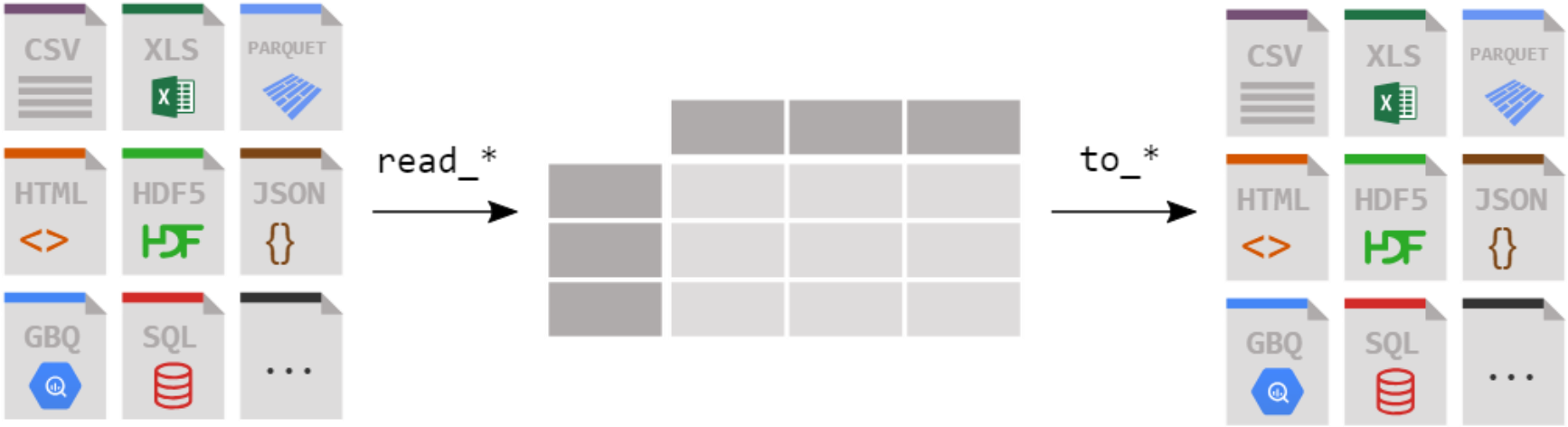
```
import pandas as pd

df = pd.DataFrame( { "Name": [ "Braund, Mr. Owen Harris",
                               "Allen, Mr. William Henry",
                               "Bonnell, Miss. Elizabeth" ],
                     "Age": [22, 35, 58],
                     "Sex": ["male", "male", "female"] })
```

	Name	Age	Sex
0	Braund, Mr. Owen Harris	22	male
1	Allen, Mr. William Henry	35	male
2	Bonnell, Miss. Elizabeth	58	female

DataFrame I/O

- Podemos crear / exportar un DataFrame desde / hacia cualquier fuente de datos en formato tabla



psycopg – PostgreSQL desde Python

- Consulta SQL para obtener fichero CSV de animales

```
import psycopg
import pandas as pd
from shapely.geometry import shape
import datetime

with psycopg.connect("dbname=mtopeniot user=crate") as conn:

    with conn.cursor() as cur:
        cur.execute("SELECT entity_id, time_index, heartrate, location, status \
                     FROM mtopeniot.etdevice ORDER BY time_index")
        tuples = cur.fetchall()

        df = pd.DataFrame(data=tuples, columns=['entity_id', 'time_index’,
                                                'heartrate', 'location', 'status'])
        df['time_index'] = df['time_index'].apply(datetime.datetime.isoformat)
        df['location'] = df['location'].apply(shape)
        df.to_csv('animals.csv', index=False)
```

GeoDataFrame

- DataFrame + columna de geometría

	BoroCode	BoroName	Shape_Leng	Shape_Area	geometry
0	5	Staten Island	330470.010332	1.623820e+09	MULTIPOLYGON (((970217.022 145643.332, 970227....
1	4	Queens	896344.047763	3.045213e+09	MULTIPOLYGON (((1029606.077 156073.814, 102957...
2	3	Brooklyn	741080.523166	1.937479e+09	MULTIPOLYGON (((1021176.479 151374.797, 102100...
3	1	Manhattan	359299.096471	6.364715e+08	MULTIPOLYGON (((981219.056 188655.316, 980940....
4	2	Bronx	464392.991824	1.186925e+09	MULTIPOLYGON (((1012821.806 229228.265, 101278...

GeoJSON

- Forma general: FeatureCollection

```
geojson_data = {
  "type": "FeatureCollection",
  "features": [ { "properties": {"name": "Alabama"},
                  "id": "AL",
                  "type": "Feature",
                  "geometry": { "type": "Polygon",
                                "coordinates": [[[-87.36, 35.00], ...]] }
                },
                { "properties": {"name": "Alaska"},
                  "id": "AK",
                  "type": "Feature",
                  "geometry": { "type": "MultiPolygon",
                                "coordinates": [[[[-13.60, 55.12], ...]]]}
                },
                ...
              ] }
```

GeoJSON FeatureCollection – GeoDataFrame – Lista entidades NGSI

- Representaciones análogas
  - Una Feature ←→ una fila del GeoDataFrame
    - *id* ←→ *index*
    - *properties* ←→ *data*
    - *geometry* ←→ *geometry*

  - Una Feature ←→ una entidad NGSI
    - *id* ←→ *id*
    - *properties* ←→ *properties*
    - *geometry* ←→ *location*

```
entity = {
  "type": "State",
  "id": "AL",
  "name": { "type": "Text",
            "value": "Alabama"},
  "location": {
      "type": "geo:json",
      "value": {
         "type": "Polygon",
         "coordinates":
            [[[-87.36, 35.00], ...]] } }
}
```

Plotly

- Librería JS con *bindings* para Python y otros lenguajes
- Módulo ***plotly.express*** *(px)*
  - Permite crear gráficas rápidamente
  - Ej.: `fig = px.scatter(x=[0, 1, 2, 3, 4], y=[0, 1, 4, 9, 16])`
- Módulo ***plotly.graph_objects*** *(go)*
  - Permite un control completo sobre los elementos del gráfico
  - Ej.:
```
fig = go.Figure(data=go.Scatter(
        x=[1, 2, 3, 4],
        y=[10, 11, 12, 13]))
```
- En ambos casos obtenemos un objeto *go*
  - Se puede empezar con *px* para crear el gráfico y después usar *go* para manipularlo

Figuras

- Una figura se **representa equivalentemente** mediante
  - Un **diccionario** (bajo nivel). Ej.:

```
fig = { "data": [{"type": "bar",
                  "x": [1, 2, 3], "y": [1, 3, 2]}],
        "layout": {"title": {"text": "Figure specified by a Python dict"}}}
```

  - Un **objeto *go***
    - Ej.:

```
fig = go.Figure(
    data=[go.Bar(x=[1, 2, 3], y=[1, 3, 2])],
    layout=go.Layout(
        title=go.layout.Title(text="Figure specified by a Graph Object")))
```

    - Ej.:

```
dict_of_fig = {
    "data": [{"type": "bar",
              "x": [1, 2, 3], "y": [1, 3, 2]}],
    "layout": {"title": {
        "text": "Figure specified by a Graph Object with a dict"}} }
fig = go.Figure(dict_of_fig)
```

Figuras (II)

- Los **objetos *go*** tienen **muchas ventajas**, entre otras:
  - **Acceso** a sus propiedades tanto mediante *figure['property']* como **mediante *figure.property***
  - Actualizar figura ***.update_layout()*** *y* ***add_trace()***
  - Notación ***magic_underscore***
  - Volcado ***.to_json()*** y ***.to_dict()***
- Los atributos de un objeto *go* se pueden fijar mediante:
  - Directamente. Ej.:

```
fig.layout.title.font.family="Open Sans"
```

  - Método *update* y notación *magic_underscore*. Ej.:

```
fig.update_layout(title_font_family="Open Sans")
```

Figuras (III)

- Métodos *.update_()*
  - ***.update_layout()***
  - ***.update_traces()***
  - ***.update_xaxes(), .update_yaxis()***
  - *.update_layout_images()*
  - *.update_annotations()*
  - *.update_shapes()*
- **Encadenamiento** de métodos *.update_()*

```
fig = px.scatter(x=[0, 1, 2, 3, 4], y=[0, 1, 4, 9, 16],
                 title="Chaining Multiple Figure Operations")
 .update_layout(title_font_size=24)
 .update_xaxes(showgrid=False)
 .update_traces(
     line=dict(dash="dot", width=4),
     selector=dict(type="scatter", mode="lines"))
```

Folium

- *Binding* Python de Leaflet JS
- Visualización de mapas

Folium.GeoJSON

- Cargar una capa a partir de un GeoJSON

```
geojson_data = {
  "type": "FeatureCollection",
  "features": [ { "properties": {"name": "Alabama"},
                  "id": "AL",
                  "type": "Feature",
                  "geometry": { "type": "Polygon",
                                "coordinates": [[[-87.36, 35.00], ...]] } },
                { "properties": {"name": "Alaska"},
                  "id": "AK",
                  "type": "Feature",
                  "geometry": { "type": "MultiPolygon",
                                "coordinates": [[[[-131.60, 55.12], ... ]]] } },
                ...
              ] }
```

```
m = folium.Map()
folium.GeoJson(geo_json_data).add_to(m)
```

Folium.GeoJSON y GeoPandas

- Cargar una capa a partir de un GeoDataFrame

```
m = folium.Map()
folium.GeoJson(gdf).add_to(m)
```

- Opciones de *folium.GeoJSON()*

```
folium.GeoJson(gdf,
    name="Subway Stations",
    marker=folium.Circle(radius=4, ... ),
    tooltip=folium.GeoJsonTooltip(fields=["name", "line", "notes"]),
    popup=folium.GeoJsonPopup(fields=["name", "line", "href", "notes"]),
    style_function=lambda x: {
        "fillColor": colors[x['properties']['service_level']],
        "radius": (x['properties']['lines_served'])*30 },
    highlight_function=lambda x: {"fillOpacity": 0.8},
    zoom_on_click=True,
).add_to(m)
```